

Computer Graphics

Lecture 04

Filling Area

Polygon Representation

The polygon can be represented by listing its n vertices in an ordered list.

$$P = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}.$$

The polygon can be displayed by drawing a line between (x_1, y_1) , and (x_2, y_2) , then a line between (x_2, y_2) , and (x_3, y_3) , and so on until the end vertex. In order to close up the polygon, a line between (x_n, y_n) , and (x_1, y_1) must be drawn.

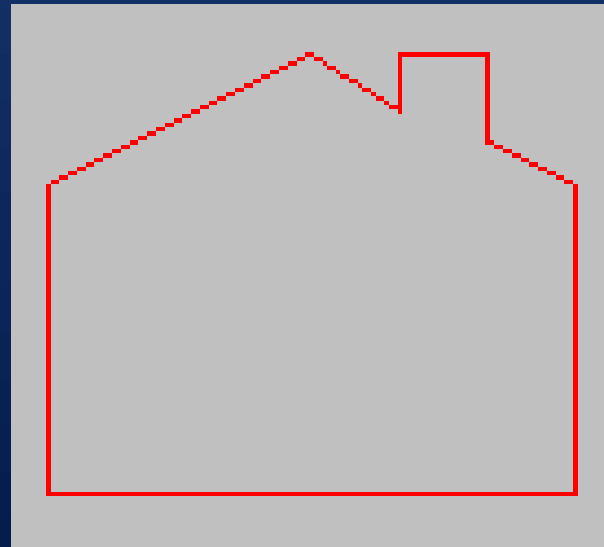
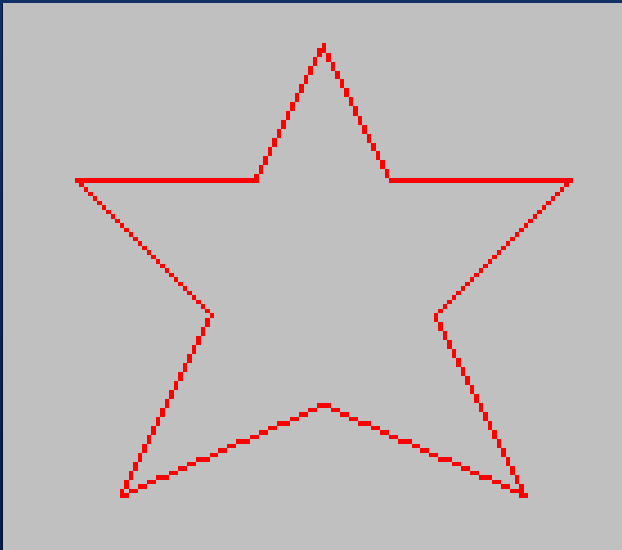
One problem with this representation is that if we wish to translate the polygon, it is necessary to apply the translation transformation to each vertex in order to obtain the translated polygon.

Polygon Types

Polygons can be divided into three basic types: **convex**, **concave**, and **complex**.

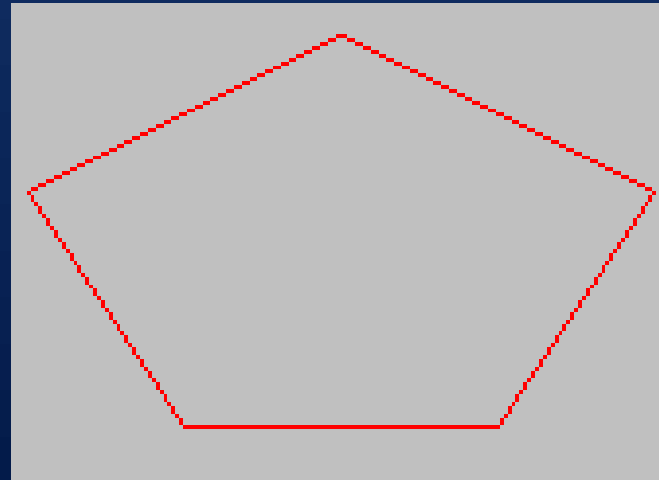
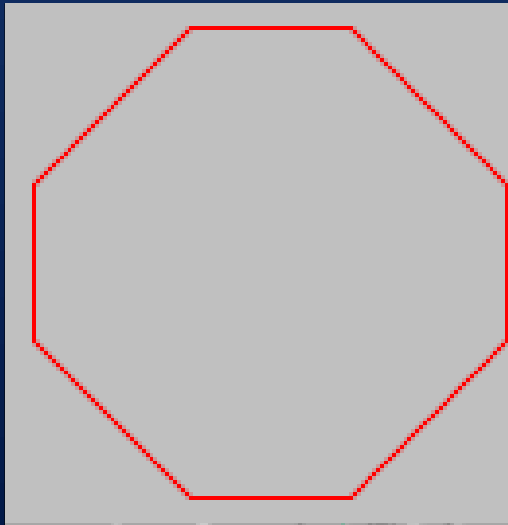
Concave

The polygon edges may also touch each other, but they may not cross one another.



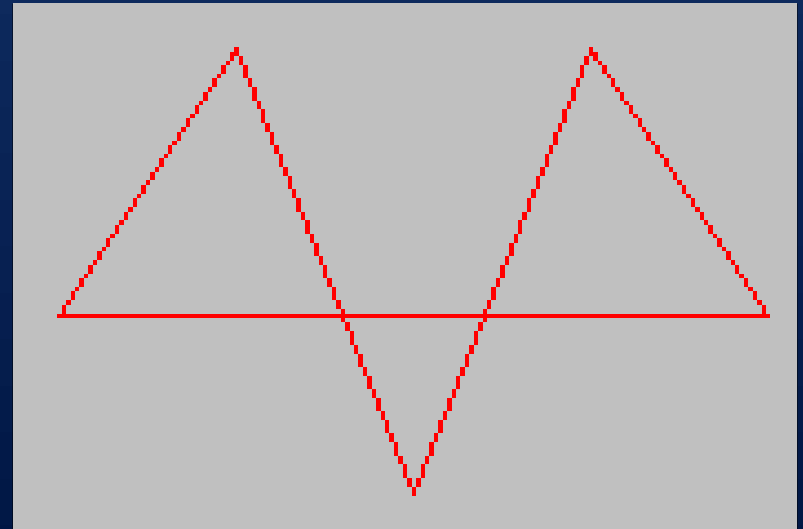
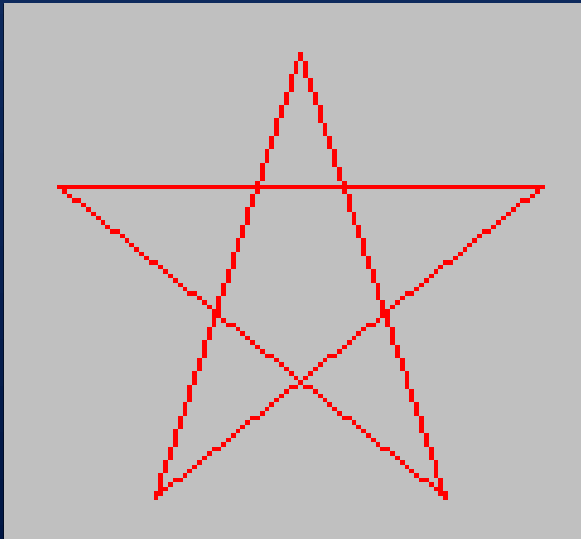
Convex

Does a straight line connecting ANY two points that are inside the polygon intersect any edges of the polygon?



Complex

Complex polygons are basically concave polygons that may have self-intersecting edges. The complexity arises while distinguishing which side is inside the polygon when filling it.



Parity Requirement

In order to determine which pixels are inside the polygon, the odd-parity rule is used within the scan-line polygon fill algorithm.

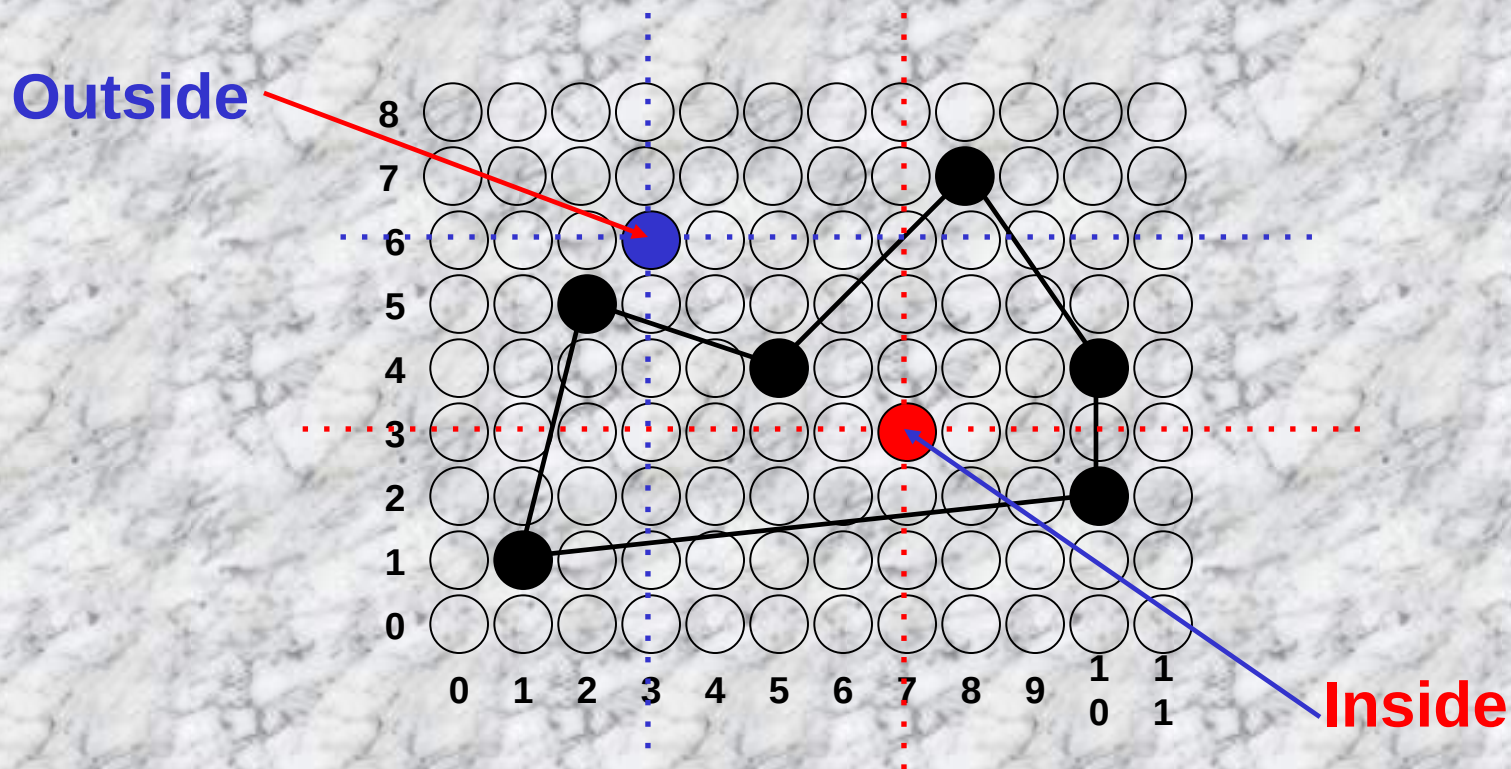
Parity

Draw a line segment from any point that lies outside the polygon to a point P.

Count the number of edges that the line crosses. If the number of polygon edges crossed is **odd**, then P lies within the polygon. Whereas, if the number of edges is even, then P lies outside the polygon.

Inside-Outside Tests

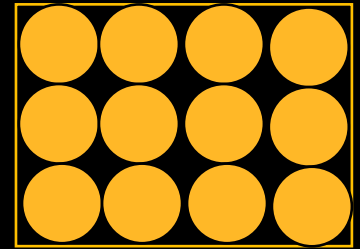
The point is considered to be **interior** if the number of intersections between the line and the polygon edges is **odd**. Otherwise, The point is exterior point.



Polygon Filling

- ◆ **What is Polygon filling?**

→ Highlight all the pixels in the polygon with any color other than the background.



- ◆ **Types of Polygon filling (2 approaches)**

- ◆ **Seed Fill:**

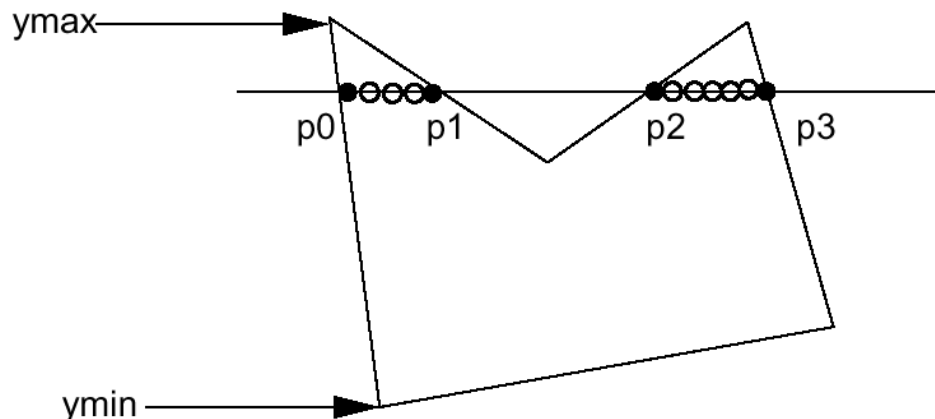
- **Boundary Filling:**

- **Flood Filling:** Select interior point - Paint outwards

- ◆ **Scan-line** : Scan lines between Boundaries and color the pixels that lies inside the polygon.

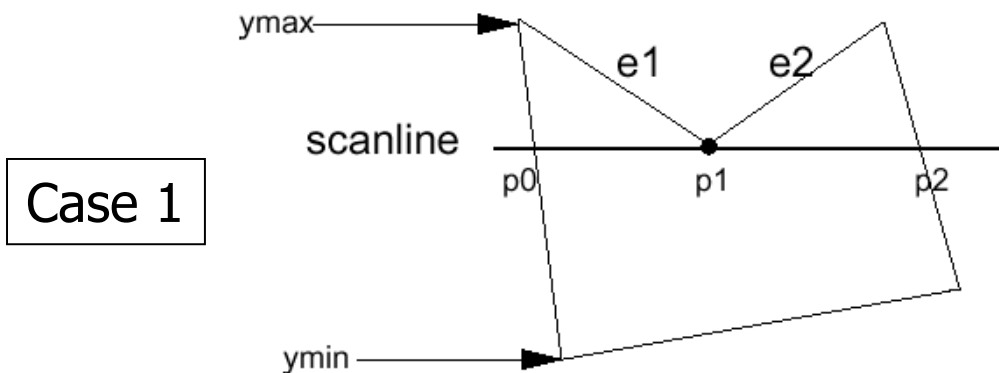
Scanline Fill Algorithm

- Intersect scanline with polygon edges
- Fill between pairs of intersections
- Basic algorithm:
For $y = y_{\min}$ to $y_{\max}$
 - 1) intersect scanline y with each edge
 - 2) sort intersections by increasing x [p_0, p_1, p_2, p_3]
 - 3) fill pairwise ($p_0 \rightarrow p_1$, $p_2 \rightarrow p_3$, ...)



Spatial Handling (cont.)

- Intersection is an edge end point



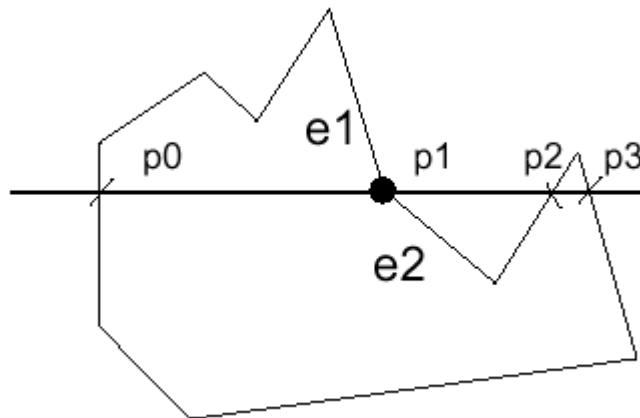
Intersection points: (p_0 , p_1 , p_2) ???

-> (p_0, p_1, p_1, p_2) so we can still fill pairwise

-> In fact, if we compute the intersection of the scanline with edge e_1 and e_2 separately, we will get the intersection point p_1 twice. Keep both of the p_1 .

Spacial Handling (cont.)

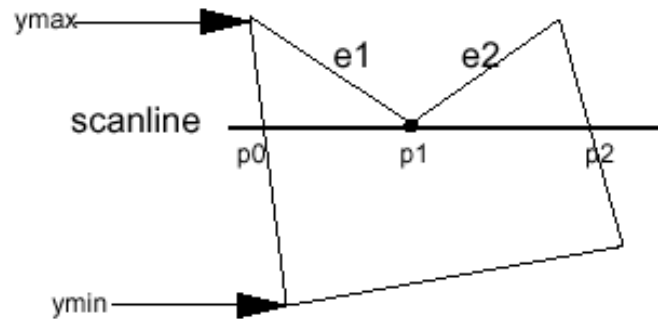
Case 2



However, in this case we don't want to count p1 twice (p0,p1,p1,p2,p3), otherwise we will fill pixels between p1 and p2, which is wrong.

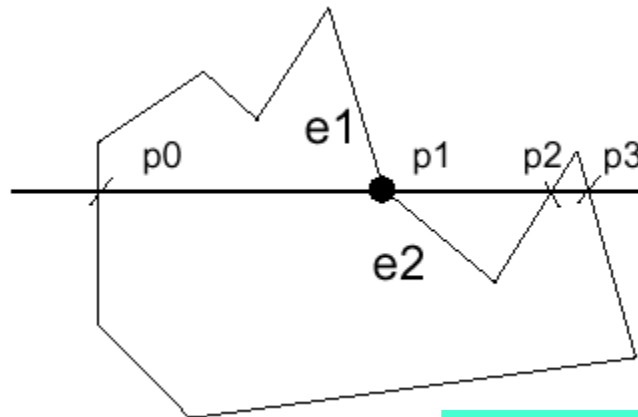
Spatial Handling (cont.)

Summary: If the intersection is the ymin of the edge's endpoint, count it. Otherwise, don't.



Yes, count $p1$ for both $e1$ and $e2$

Both edges are in the same side of the scan line



No, don't count $p1$ for the edge $e2$

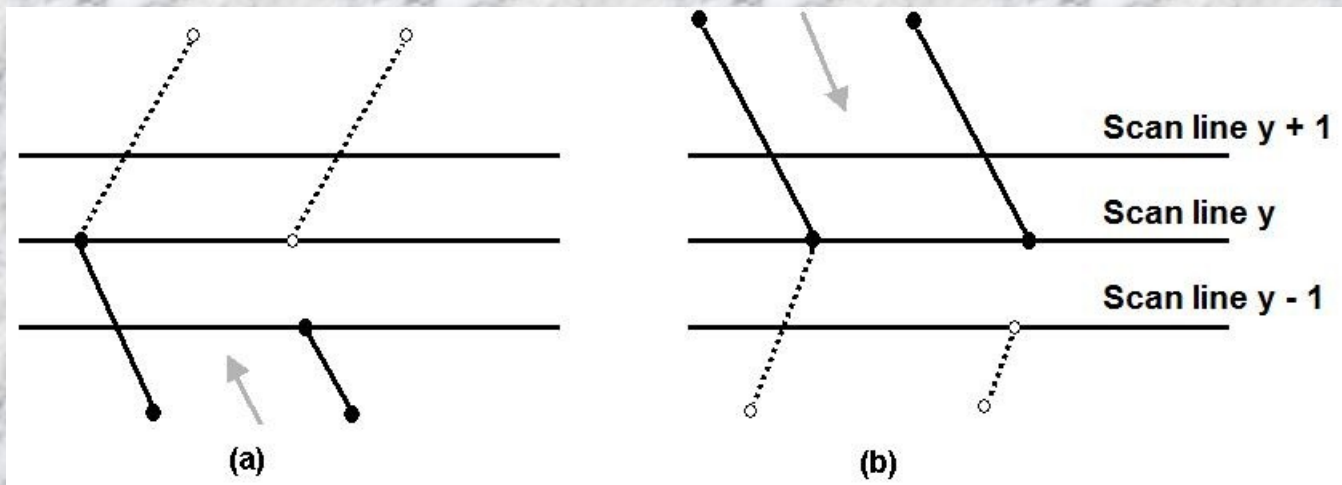
Both edges are in the opposite side of the scan line

Fix a new start point for $e2$

The Scan-Line Polygon Fill Algorithm

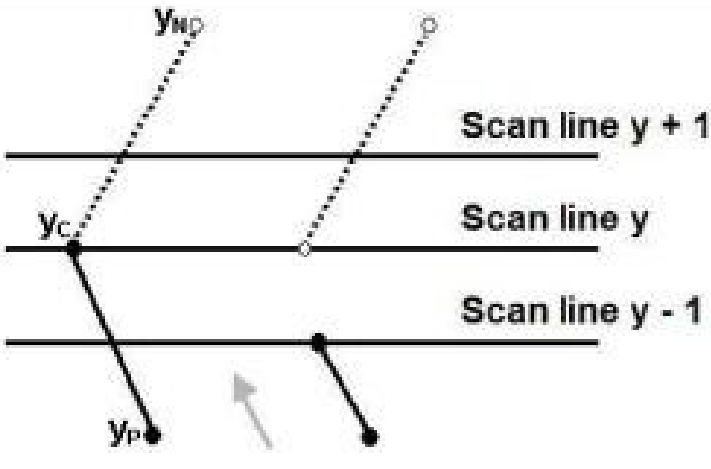
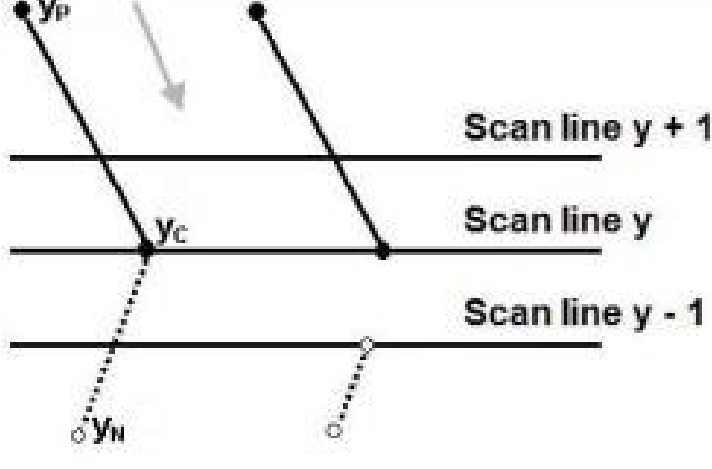
Dealing with vertices

- When the endpoint **y** coordinates of the two edges are monotonously **increasing**, the **y** value of the upper endpoint for the **current edge** is decreased by one (a)
- When the endpoint **y** values are monotonously **decreasing**, the **y** value of the **next edge** is decreased by one (b)



The Scan-Line Polygon Fill Algorithm

(Example)

	
If $y_P < y_C < y_N$ Then y_C is decreased by one. The new edges become $(x_P, y_P) \rightarrow (x'_C, y_C - 1)$ and $(x_C, y_C) \rightarrow (x_N, y_N)$	If $y_P > y_C > y_N$ Then y_C is decreased by one. The new edges become $(x_P, y_P) \rightarrow (x_C, y_C)$ and $(x'_C, y_C - 1) \rightarrow (x_N, y_N)$
$m = (y_P - y_C) / (x_P - x_C)$ $x'_C = x_P + (1/m)(y_C - 1 - y_P)$	$m = (y_N - y_C) / (x_N - x_C)$ $x'_C = x_N + (1/m)(y_C - 1 - y_N)$

N.B. $x_2 = x_1 + 1/m(y_2 - y_1)$

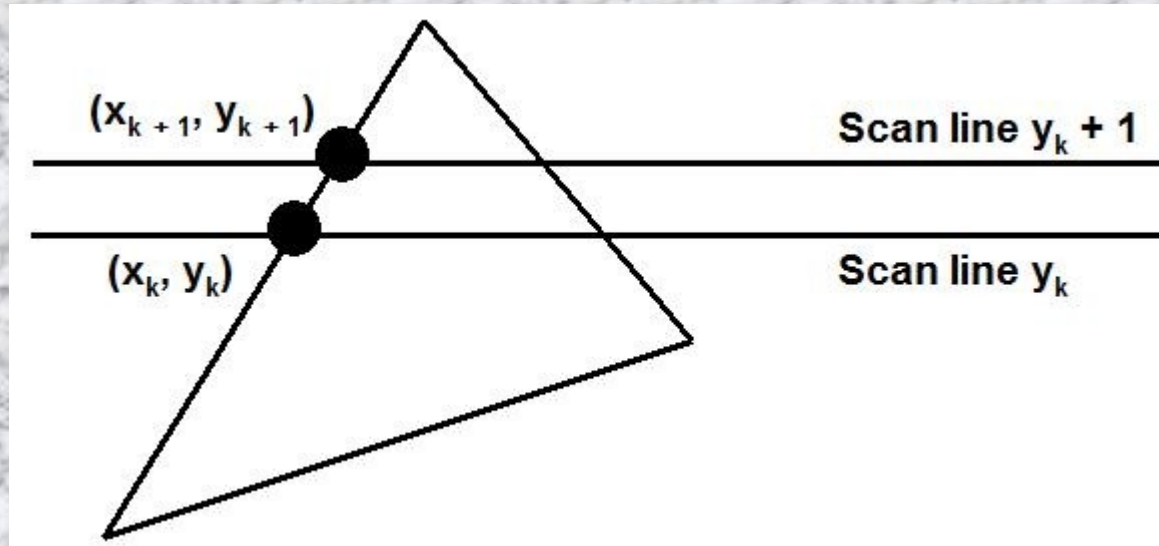
The Scan-Line Polygon Fill Algorithm

Determining Edge Intersections

$$m = (y_{k+1} - y_k) / (x_{k+1} - x_k)$$

$$y_{k+1} - y_k = 1$$

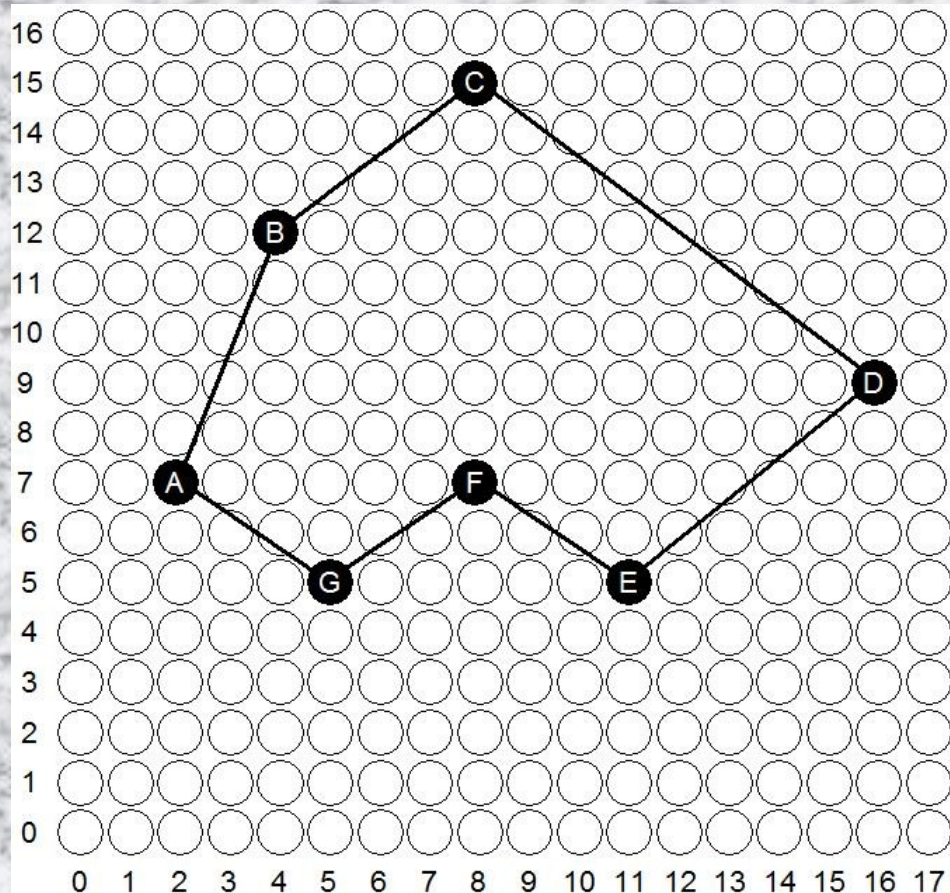
$$x_{k+1} = x_k + 1/m$$



The Scan-Line Polygon Fill Algorithm

(Example) Polygon = {A, B, C, D, E, F, G}

Polygon = {(2, 7), (4, 12), (8, 15), (16, 9), (11, 5), (8, 7), (5, 5)}



Edge Table						
#	Edge		1/m	ymin	x	y _{max}
0	A (2, 7)	B (4, 12)	2/5	7	2	12
1	B (4, 12)	C (8, 15)	4/3	12	4	15
2	C (8, 15)	D (16, 9)	-8/6	9	16	15
3	D (16, 9)	E (11, 5)	5/4	5	11	9
4	E (11, 5)	F (8, 7)	-3/2	5	11	7
5	F (8, 7)	G (5, 5)	3/2	5	5	7
6	G (5, 5)	A (2, 7)	-3/2	5	5	7

The Scan-Line Polygon Fill Algorithm(Example)

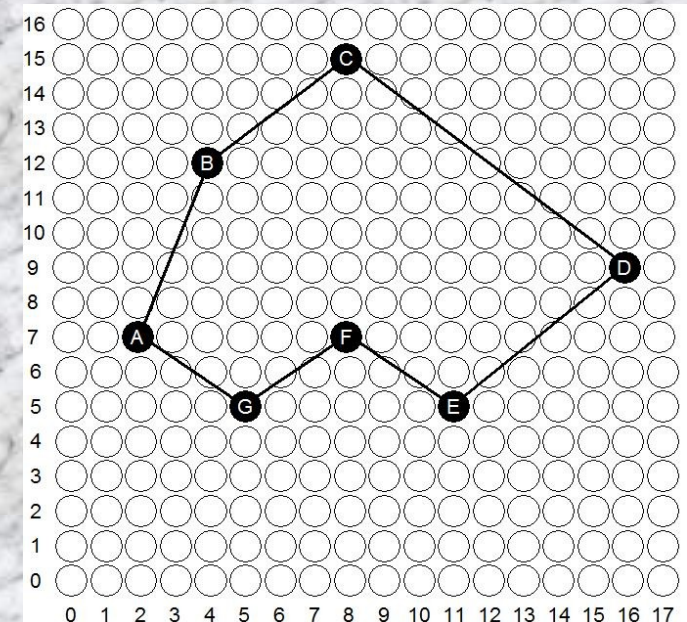
Previous Vertex	Current Vertex	Next Vertex	$y_P ? y_C ? y_N$	Current Vertex Type	Action
G (5, 5)	A (2, 7)	B (4, 12)	$y_P < y_C < y_N$	Not local extremum	Split A
A (2, 7)	B (4, 12)	C (8, 15)	$y_P < y_C < y_N$	Not local extremum	Split B
B (4, 12)	C (8, 15)	D (16, 9)	$y_P < y_C > y_N$	Local Maximum	None
C (8, 15)	D (16, 9)	E (11, 5)	$y_P > y_C > y_N$	Not local extremum	Split D
D (16, 9)	E (11, 5)	F (8, 7)	$y_P > y_C < y_N$	Local Minimum	None
E (11, 5)	F (8, 7)	G (5, 5)	$y_P < y_C > y_N$	Local Maximum	None
F (8, 7)	G (5, 5)	A (2, 7)	$y_P > y_C < y_N$	Local Minimum	None

- Vertex A should be split into two vertices A' ($x_{A'}$, 6) and A(2, 7)

$$m = (5 - 7) / (5 - 2) = -2/3$$

$$x'_{A'} = 5 + (-2/3)(7 - 1 - 5) = 7/2 = 3.5 \approx 4$$

The vertex A is split to A' (4, 6) and A(2, 7)



The Scan-Line Polygon Fill Algorithm(Example)

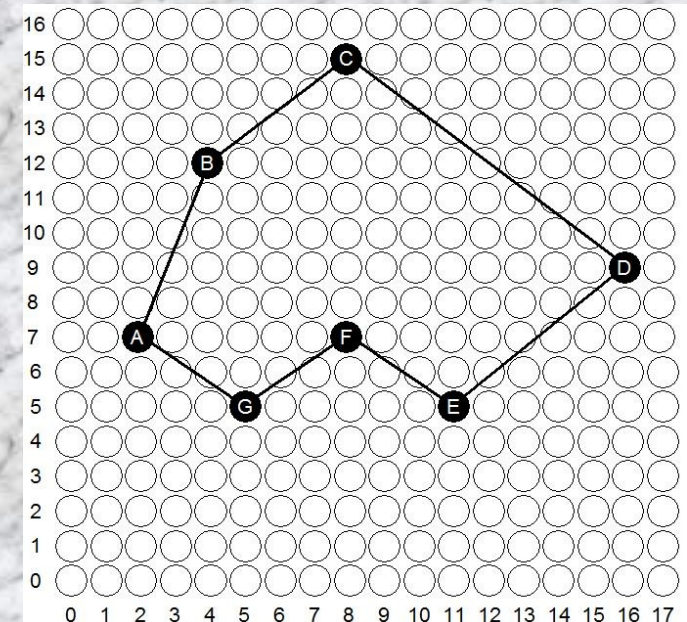
Previous Vertex	Current Vertex	Next Vertex	$y_P ? y_C ? y_N$	Current Vertex Type	Action
G (5, 5)	A (2, 7)	B (4, 12)	$y_P < y_C < y_N$	Not local extremum	Split A
A (2, 7)	B (4, 12)	C (8, 15)	$y_P < y_C < y_N$	Not local extremum	Split B
B (4, 12)	C (8, 15)	D (16, 9)	$y_P < y_C > y_N$	Local Maximum	None
C (8, 15)	D (16, 9)	E (11, 5)	$y_P > y_C > y_N$	Not local extremum	Split D
D (16, 9)	E (11, 5)	F (8, 7)	$y_P > y_C < y_N$	Local Minimum	None
E (11, 5)	F (8, 7)	G (5, 5)	$y_P < y_C > y_N$	Local Maximum	None
F (8, 7)	G (5, 5)	A (2, 7)	$y_P > y_C < y_N$	Local Minimum	None

- Vertex B should be split into two vertices B' ($x_{B'}$, 11) and B(4, 12)

$$m = (7 - 12) / (2 - 4) = 5/2$$

$$x'_A = 2 + (2/5)(12 - 1 - 7) = 18/5 = 3.6 \approx 4$$

The vertex B is split to B' (4, 11) and B(4, 12)



The Scan-Line Polygon Fill Algorithm(Example)

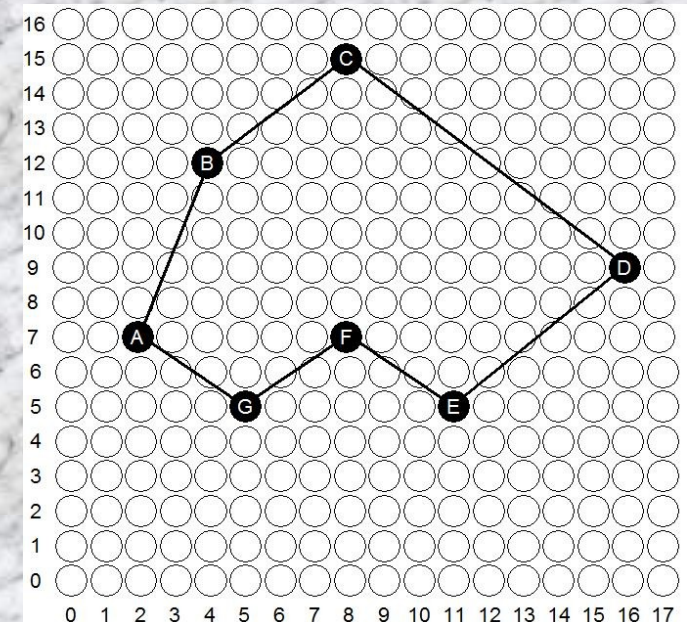
Previous Vertex	Current Vertex	Next Vertex	$y_P ? y_C ? y_N$	Current Vertex Type	Action
G (5, 5)	A (2, 7)	B (4, 12)	$y_P < y_C < y_N$	Not local extremum	Split A
A (2, 7)	B (4, 12)	C (8, 15)	$y_P < y_C < y_N$	Not local extremum	Split B
B (4, 12)	C (8, 15)	D (16, 9)	$y_P < y_C > y_N$	Local Maximum	None
C (8, 15)	D (16, 9)	E (11, 5)	$y_P > y_C > y_N$	Not local extremum	Split D
D (16, 9)	E (11, 5)	F (8, 7)	$y_P > y_C < y_N$	Local Minimum	None
E (11, 5)	F (8, 7)	G (5, 5)	$y_P < y_C > y_N$	Local Maximum	None
F (8, 7)	G (5, 5)	A (2, 7)	$y_P > y_C < y_N$	Local Minimum	None

- Vertex **D** should be split into two vertices **D**(16, 9) and **D'**(x_D , 8)

$$m = (5 - 9) / (11 - 16) = 4/5$$

$$x'_D = 11 + (5/4)(9 - 15) = 59/4 = 14.75 \approx 15$$

The vertex **D** is split to **D**(16, 9) and **D'**(15, 8)



The Scan-Line Polygon Fill Algorithm(Example)

Modified Edge Table						
#	Edge		1/m	y _{min}	x	y _{max}
0	A (2, 7)	B' (4, 11)	2/5	7	2	11
1	B (4, 12)	C (8, 15)	4/3	12	4	15
2	C (8, 15)	D (16, 9)	- 8/6	9	16	15
3	D' (15, 8)	E (11, 5)	5/4	5	11	8
4	E (11, 5)	F (8, 7)	- 3/2	5	11	7
5	F (8, 7)	G (5, 5)	3/2	5	5	7
6	G (5, 5)	A' (4, 6)	- 3/2	5	5	6

Activation Table											
y	5	6	7	8	9	10	11	12	13	14	15
Activated Edge #s	3, 4, 5, 6		0		2			1			

The Scan-Line Polygon Fill Algorithm(Example)

Modified Edge Table						
#	Edge		1/m	ymin	x	y _{max}
0	A (2, 7)	B' (4, 11)	2/5	7	2	11
1	B (4, 12)	C (8, 15)	4/3	12	4	15
2	C (8, 15)	D (16, 9)	- 8/6	9	16	15
3	D' (15, 8)	E (11, 5)	5/4	5	11	8
4	E (11, 5)	F (8, 7)	- 3/2	5	11	7
5	F (8, 7)	G (5, 5)	3/2	5	5	7
6	G (5, 5)	A' (4, 6)	- 3/2	5	5	6

Edge number 0

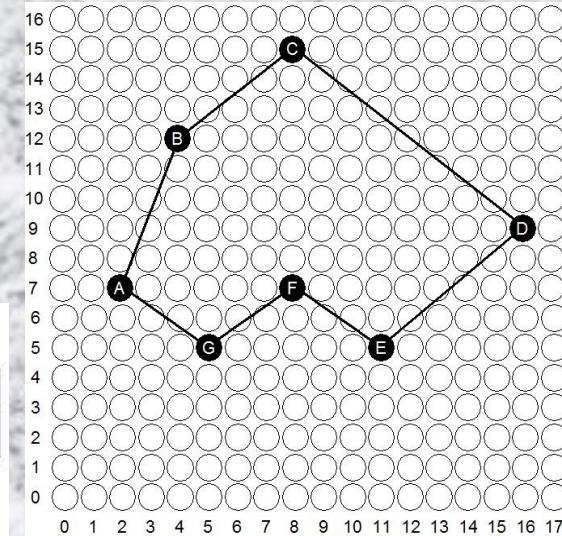
#	Edge		1/m	ymin	x	y _{max}
0	A (2, 7)	B' (4, 11)	2/5 = 0.4	7	2	11

Scan line	x-intersection
y = 7	2
y = 8	2 + 0.4 = 2.4 ~ 2
y = 9	2.4 + 0.4 = 2.8 ~ 3
y = 10	2.8 + 0.4 = 3.2 ~ 3
y = 11	4

Edge number 1

#	Edge		1/m	ymin	x	y _{max}
1	B (4, 12)	C (8, 15)	4/3 = 1.3	12	4	15

Scan line	x-intersection
y = 12	4
y = 13	4 + 1.3 = 4.3 ~ 4
y = 14	4.3 + 1.3 = 5.6 ~ 6
y = 15	8



The Scan-Line Polygon Fill Algorithm(Example)

Modified Edge Table						
#	Edge		1/m	y _{min}	x	y _{max}
0	A (2, 7)	B' (4, 11)	2/5	7	2	11
1	B (4, 12)	C (8, 15)	4/3	12	4	15
2	C (8, 15)	D (16, 9)	- 8/6	9	16	15
3	D' (15, 8)	E (11, 5)	5/4	5	11	8
4	E (11, 5)	F (8, 7)	- 3/2	5	11	7
5	F (8, 7)	G (5, 5)	3/2	5	5	7
6	G (5, 5)	A' (4, 6)	- 3/2	5	5	6

Edge number 2

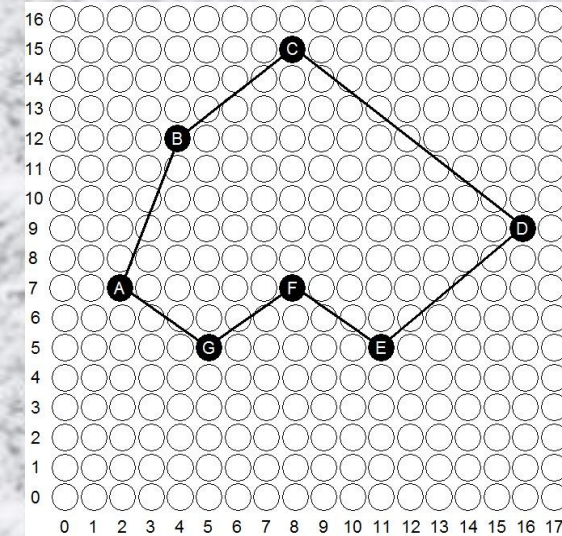
#	Edge		1/m	y _{min}	x	y _{max}
2	C (8, 15)	D (16, 9)	- 8/6 = -1.3	9	16	15

Scan line	x-intersection
y = 9	16
y = 10	16 - 1.3 = 14.7 ~ 15
y = 11	14.7 - 1.3 = 13.4 ~ 13
y = 12	13.4 - 1.3 = 12.1 ~ 12
y = 13	12.1 - 1.3 = 10.8 ~ 11
y = 14	10.8 - 1.3 = 9.5 ~ 10
y = 15	8

Edge number 3

#	Edge		1/m	y _{min}	x	y _{max}
3	D' (15, 8)	E (11, 5)	5/4 = 1.25	5	11	8

Scan line	x-intersection
y = 5	11
y = 6	11 + 1.25 = 12.25 ~ 12
y = 7	12.25 + 1.25 = 13.5 ~ 14
y = 8	15



The Scan-Line Polygon Fill Algorithm(Example)

Modified Edge Table						
#	Edge		1/m	ymin	x	y _{max}
0	A (2, 7)	B' (4, 11)	2/5	7	2	11
1	B (4, 12)	C (8, 15)	4/3	12	4	15
2	C (8, 15)	D (16, 9)	- 8/6	9	16	15
3	D' (15, 8)	E (11, 5)	5/4	5	11	8
4	E (11, 5)	F (8, 7)	- 3/2	5	11	7
5	F (8, 7)	G (5, 5)	3/2	5	5	7
6	G (5, 5)	A' (4, 6)	- 3/2	5	5	6

Edge number 4

#	Edge		1/m	ymin	x	y _{max}
4	E (11, 5)	F (8, 7)	- 3/2 = -1.5	5	11	7

Scan line	x-intersection
y = 5	11
y = 6	11 - 1.5 = 9.5 ~ 10
y = 7	8

Edge number 5

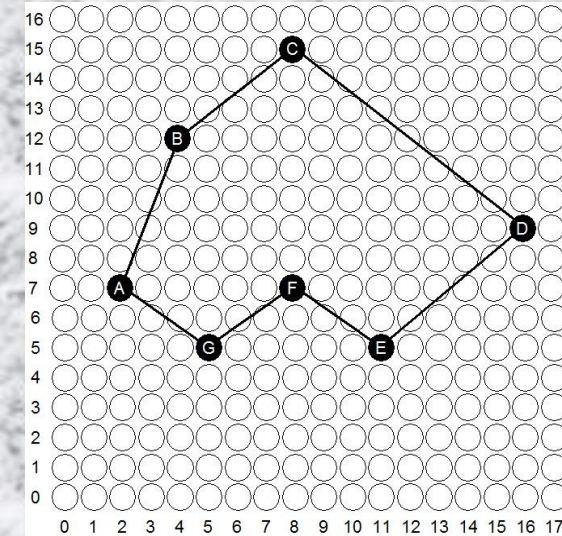
#	Edge		1/m	ymin	x	y _{max}
5	F (8, 7)	G (5, 5)	3/2 = 1.5	5	5	7

Scan line	x-intersection
y = 5	5
y = 6	5 + 1.5 = 6.5 ~ 7
y = 7	8

Edge number 6

#	Edge		1/m	ymin	x	y _{max}
6	G (5, 5)	A' (4, 6)	- 3/2 = -1.5	5	5	6

Scan line	x-intersection
y = 5	5
y = 6	4



Edge number 0

#	Edge		1/m	y _{min}	x	y _{max}
0	A (2, 7)	B' (4, 11)	2/5 = 0.4	7	2	11

Scan line	x-intersection
y = 7	2
y = 8	2 + 0.4 = 2.4 ~ 2
y = 9	2.4 + 0.4 = 2.8 ~ 3
y = 10	2.8 + 0.4 = 3.2 ~ 3
y = 11	4

Edge number 1

#	Edge		1/m	y _{min}	x	y _{max}
1	B (4, 12)	C (8, 15)	4/3 = 1.3	12	4	15

Scan line	x-intersection
y = 12	4
y = 13	4 + 1.3 = 4.3 ~ 4
y = 14	4.3 + 1.3 = 5.6 ~ 6
y = 15	8

Edge number 2

#	Edge		1/m	y _{min}	x	y _{max}
2	C (8, 15)	D (16, 9)	-8/6 = -1.3	9	16	15

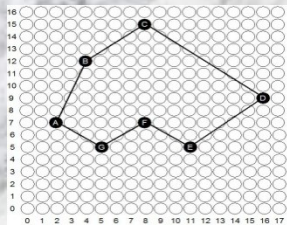
Scan line	x-intersection
y = 9	16
y = 10	16 - 1.3 = 14.7 ~ 15
y = 11	14.7 - 1.3 = 13.4 ~ 13
y = 12	13.4 - 1.3 = 12.1 ~ 12
y = 13	12.1 - 1.3 = 10.8 ~ 11
y = 14	10.8 - 1.3 = 9.5 ~ 10
y = 15	8

Edge number 3

#	Edge		1/m	y _{min}	x	y _{max}
3	D' (15, 8)	E (11, 5)	5/4 = 1.25	5	11	8

Scan line	x-intersection
y = 5	11
y = 6	11 + 1.25 = 12.25 ~ 12
y = 7	12.25 + 1.25 = 13.5 ~ 14
y = 8	15

Modified Edge Table						
#	Edge		1/m	y _{min}	x	y _{max}
0	A (2, 7)	B' (4, 11)	2/5	7	2	11
1	B (4, 12)	C (8, 15)	4/3	12	4	15
2	C (8, 15)	D (16, 9)	-8/6	9	16	15
3	D' (15, 8)	E (11, 5)	5/4	5	11	8
4	E (11, 5)	F (8, 7)	-3/2	5	11	7
5	F (8, 7)	G (5, 5)	3/2	5	5	7
6	G (5, 5)	A' (4, 6)	-3/2	5	5	6



The Scan-Line Polygon Fill Algorithm(Example)

Edge number 4

#	Edge		1/m	y _{min}	x	y _{max}
4	E (11, 5)	F (8, 7)	-3/2 = -1.5	5	11	7

Scan line	x-intersection
y = 5	11
y = 6	11 - 1.5 = 9.5 ~ 10
y = 7	8

Edge number 5

#	Edge		1/m	y _{min}	x	y _{max}
5	F (8, 7)	G (5, 5)	3/2 = 1.5	5	5	7

Scan line	x-intersection
y = 5	5
y = 6	5 + 1.5 = 6.5 ~ 7
y = 7	8

Edge number 6

#	Edge		1/m	y _{min}	x	y _{max}
6	G (5, 5)	A' (4, 6)	-3/2 = -1.5	5	5	6

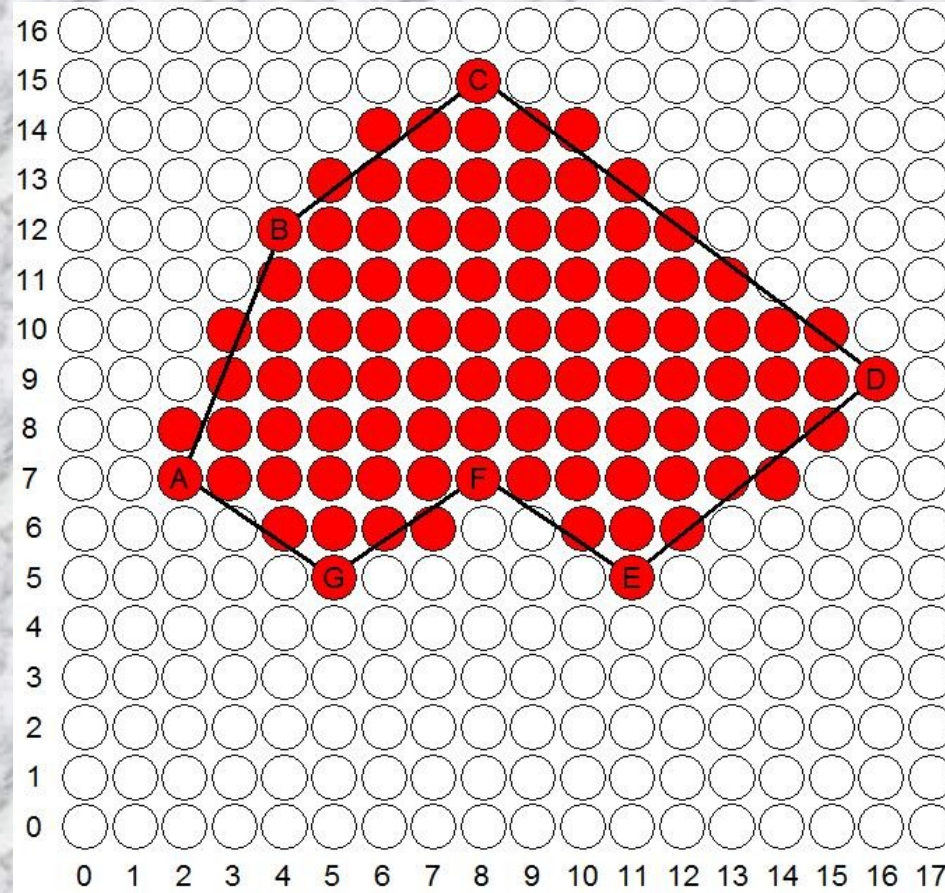
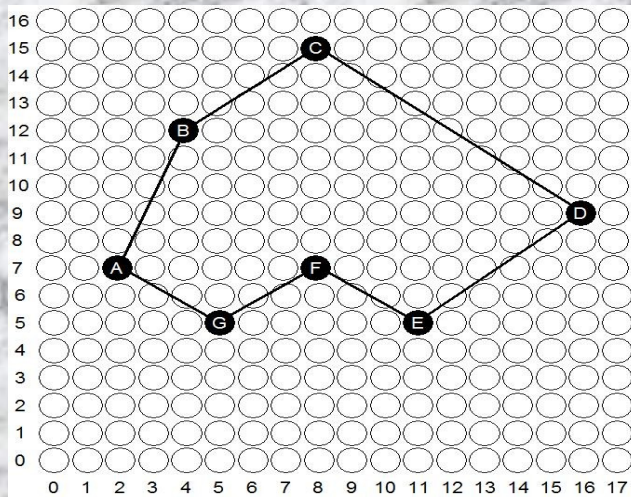
Scan line	x-intersection
y = 5	5
y = 6	4

Scan line	x-intersections							x-intersections pair Ascending order
	Edge#							
	0	1	2	3	4	5	6	
5				11	11	5	5	(5, 5), (11, 11)
6				12	10	7	4	(4, 7), (10, 12)
7	2			14	8	8		(2, 8), (8,14)
8	2			15				(2,15)
9	3		16					(3, 16)
10	3		15					(3, 15)
11	4		13					(4, 13)
12		4	12					(4,12)
13		4	11					(4, 11)
14		6	10					(6, 10)
15		8	8					(8, 8)

The Scan-Line Polygon Fill Algorithm

(Example)

Scan line	x-intersections							x-intersections pair Ascending order
	Edge#							
	0	1	2	3	4	5	6	
5				11	11	5	5	(5, 5), (11, 11)
6				12	10	7	4	(4, 7), (10, 12)
7	2			14	8	8		(2, 8), (8,14)
8	2			15				(2,15)
9	3		16					(3, 16)
10	3		15					(3, 15)
11	4		13					(4, 13)
12		4	12					(4,12)
13		4	11					(4, 11)
14		6	10					(6, 10)
15		8	8					(8, 8)



Seed Fill Algorithm

Boundary-fill: Start from an interior point and paint outward from this point till the boundary is reached; called boundary-fill.

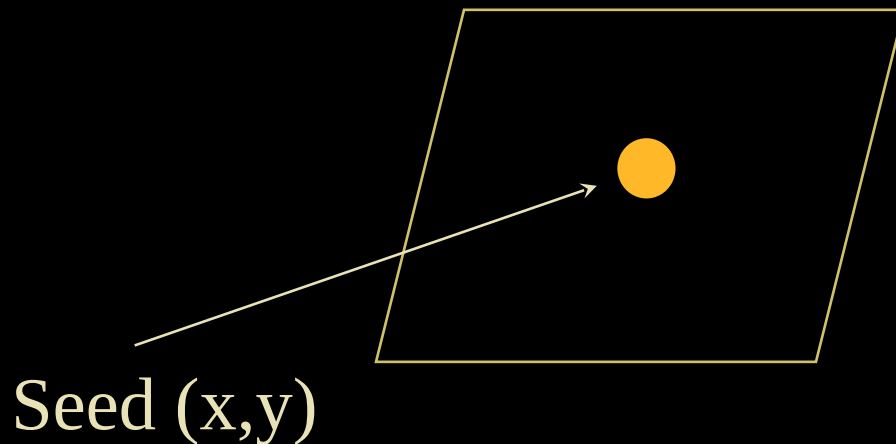
Flood-fill: Start from an interior point and start painting outward from this point till the end of cluster.

Polygon Filling

- ◆ **Seed Fill:**

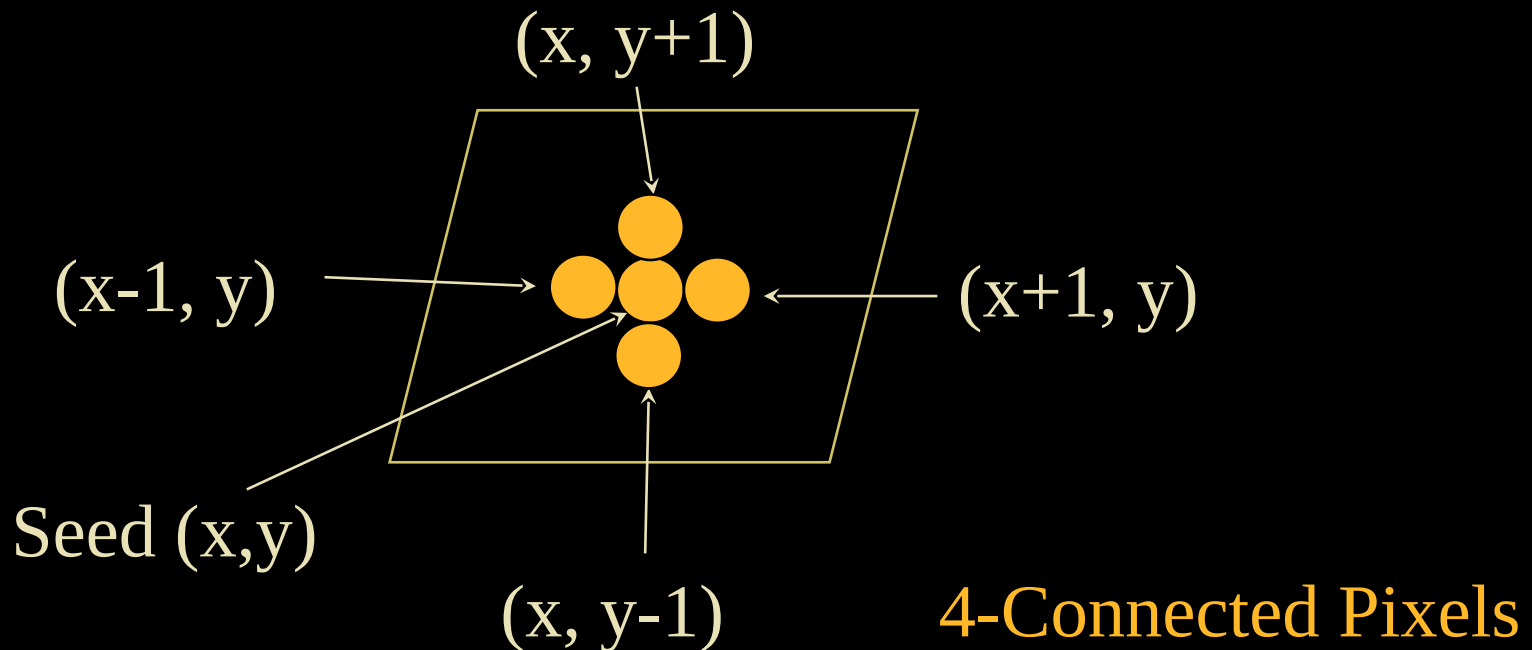
- **Boundary Filling:**

- Edges of the polygon is drawn first. Take a seed pixel (x,y) inside the polygon.
- Find the neighboring pixels until boundary is reached



Polygon Filling

- ◆ **Seed Fill:**
 - **Boundary Filling:**
 - a. Edges of the polygon is drawn first. Take a seed pixel (x,y) inside the polygon.
 - b. Find the neighboring pixels until boundary is reached.
- ◆ Two types of searching a) 4-connected b) 8-connected pixels



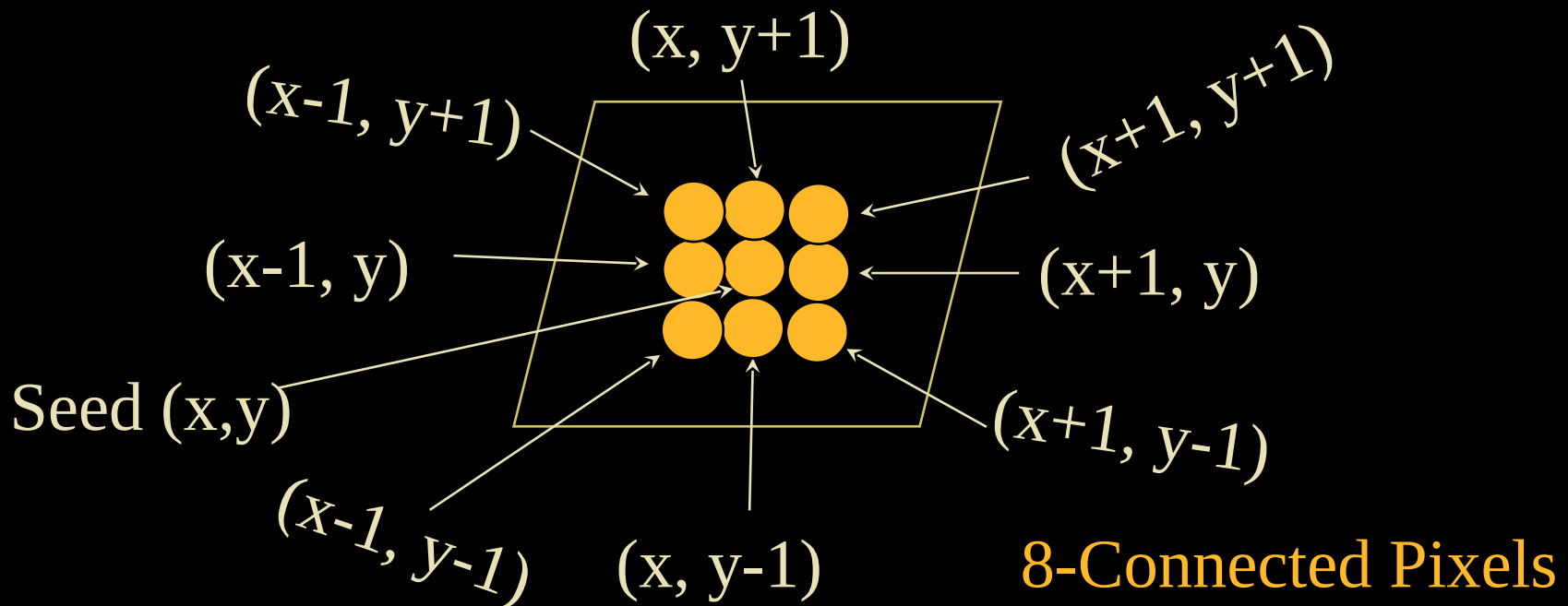
Polygon Filling

- ◆ **Seed Fill:**

- **Boundary Filling:**

- a. Edges of the polygon is drawn first. Take a seed pixel (x,y) inside the polygon.
 - b. Find the neighboring pixels until boundary is reached.

- ◆ Two types of searching a) 4-connected b) 8-connected pixels



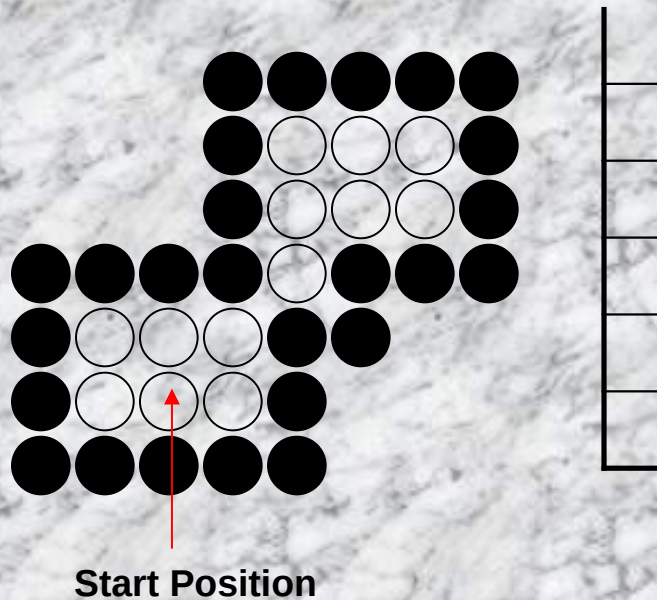
Boundary Fill Algorithm

The following steps illustrate the idea of the **recursive** boundary-fill algorithm:

- 1.** Start from an interior point.
- 2.** If the current pixel is **not already** filled and if it is not an edge point, then set the pixel with the fill color, and store its neighboring pixels (**4** or **8-connected**) in the stack for processing. Store only neighboring pixel that is **not already** filled and is not an edge point.
- 3.** Select the next pixel from the stack, and continue with step **2**.

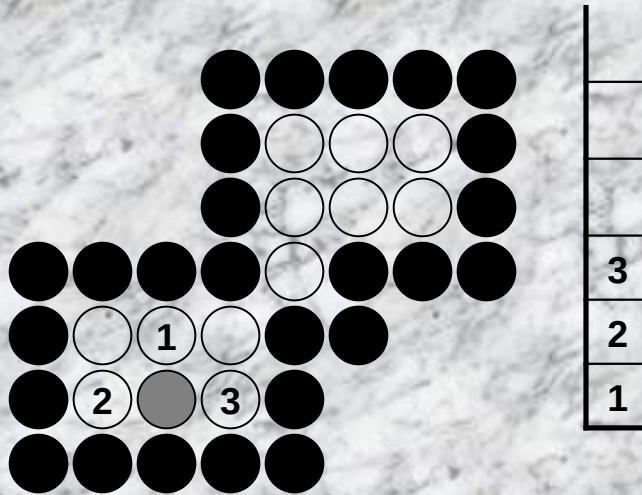
Boundary Fill Algorithm

4-connected (Example)



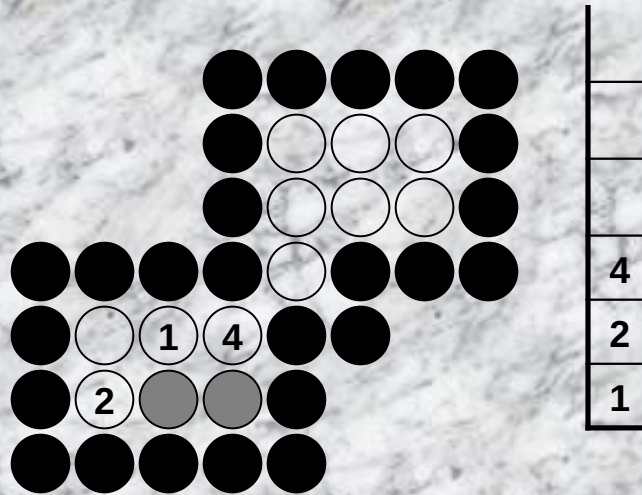
Boundary Fill Algorithm

4-connected (Example)



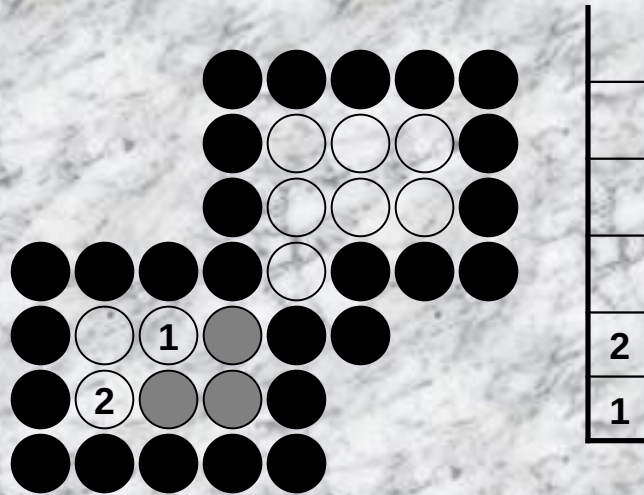
Boundary Fill Algorithm

4-connected (Example)



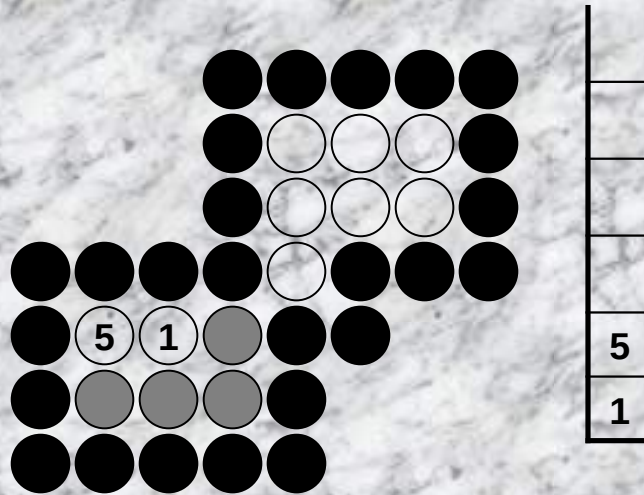
Boundary Fill Algorithm

4-connected (Example)



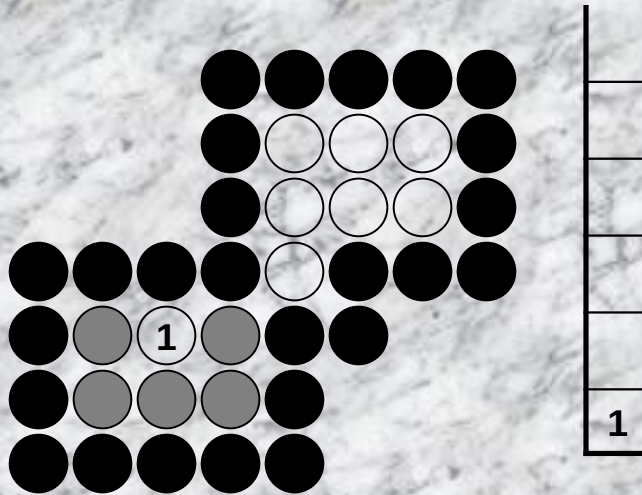
Boundary Fill Algorithm

4-connected (Example)



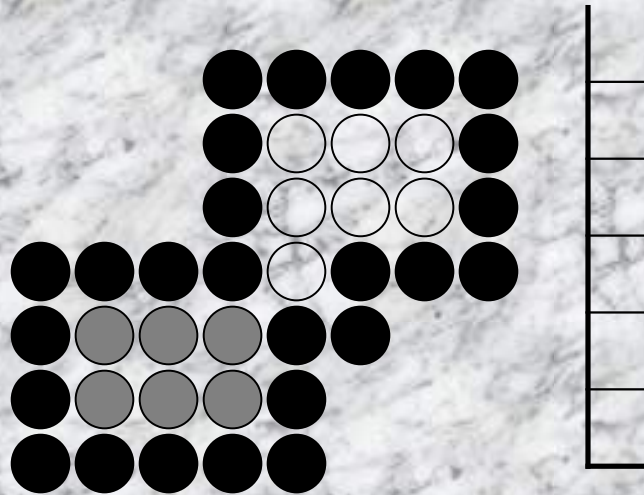
Boundary Fill Algorithm

4-connected (Example)



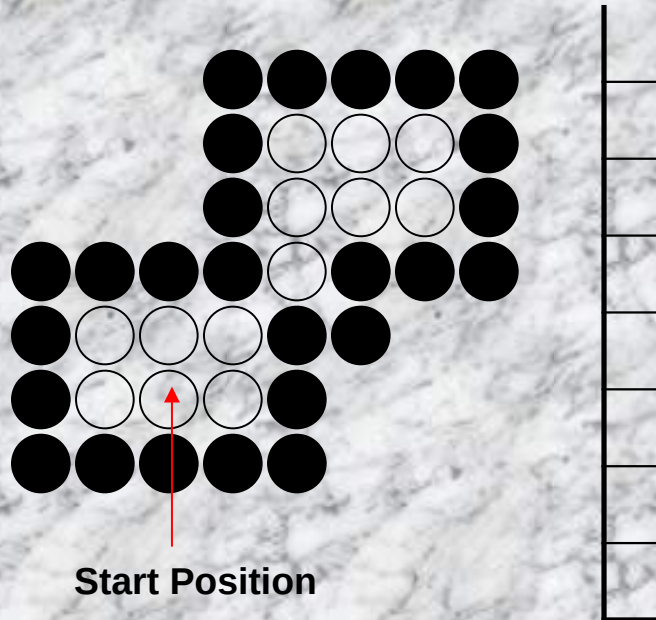
Boundary Fill Algorithm

4-connected (Example)



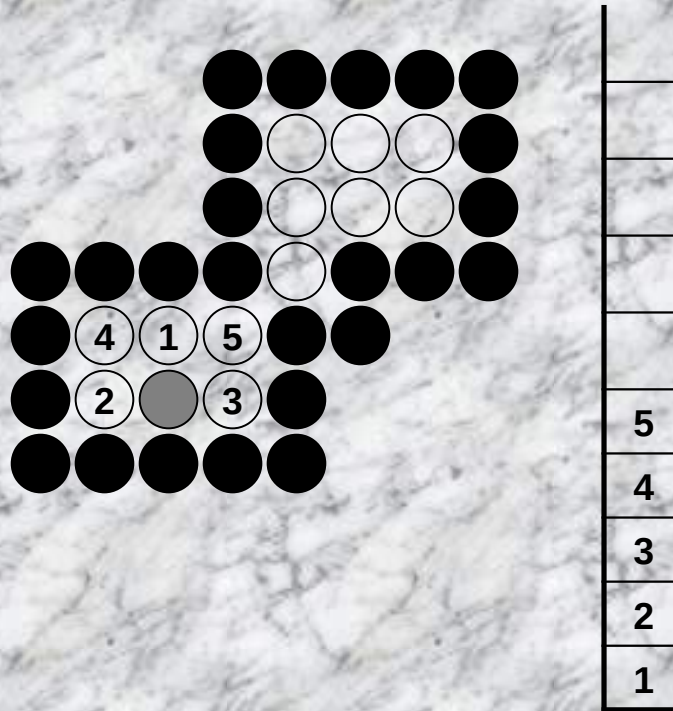
Boundary Fill Algorithm

8-connected (Example)



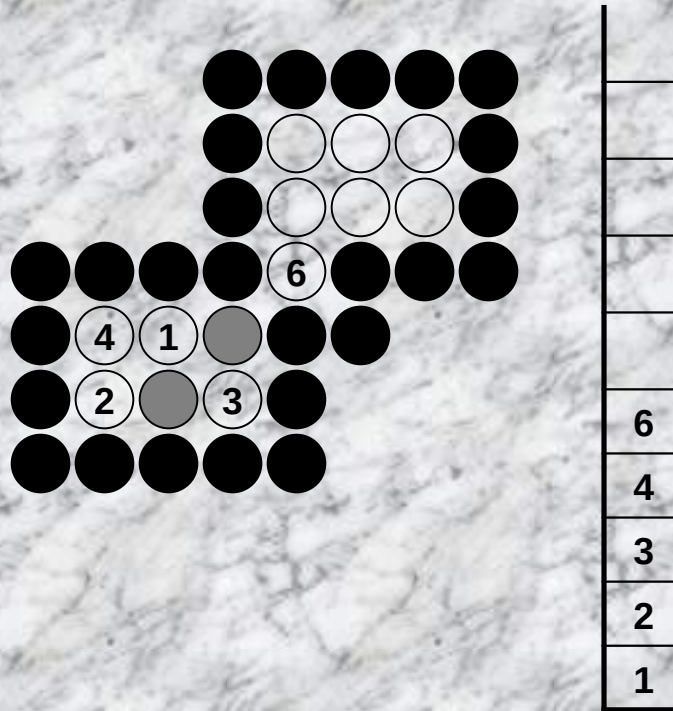
Boundary Fill Algorithm

8-connected (Example)



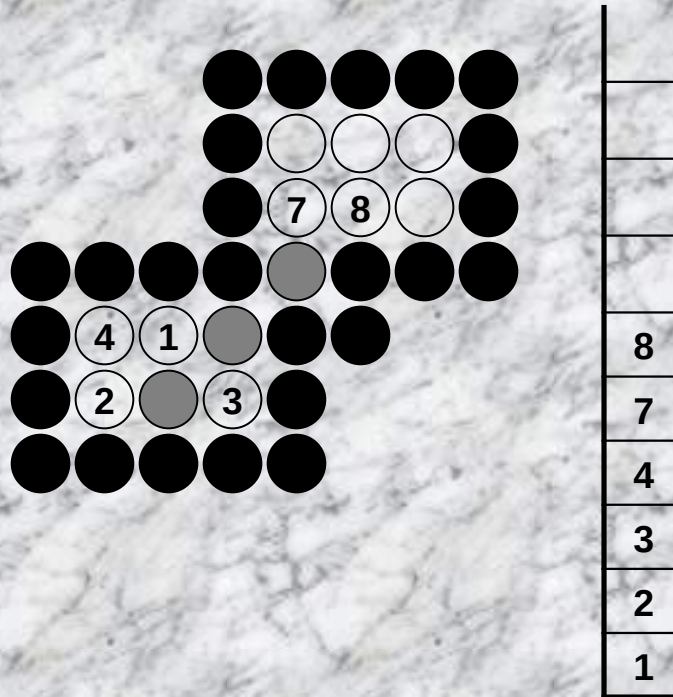
Boundary Fill Algorithm

8-connected (Example)



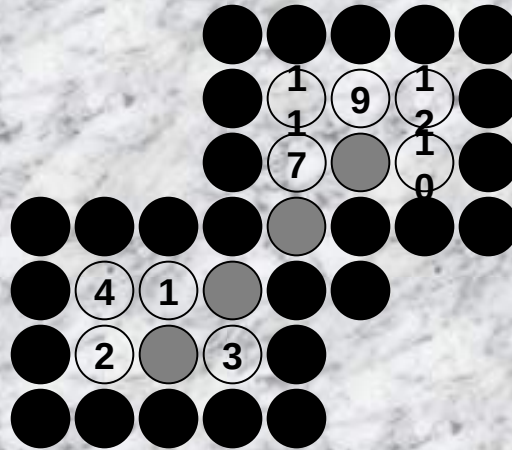
Boundary Fill Algorithm

8-connected (Example)



Boundary Fill Algorithm

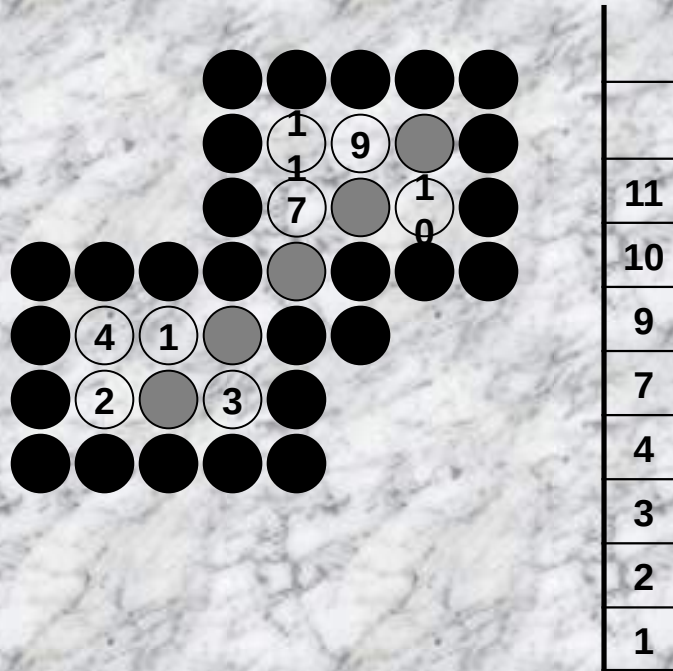
8-connected (Example)



12
11
10
9
7
4
3
2
1

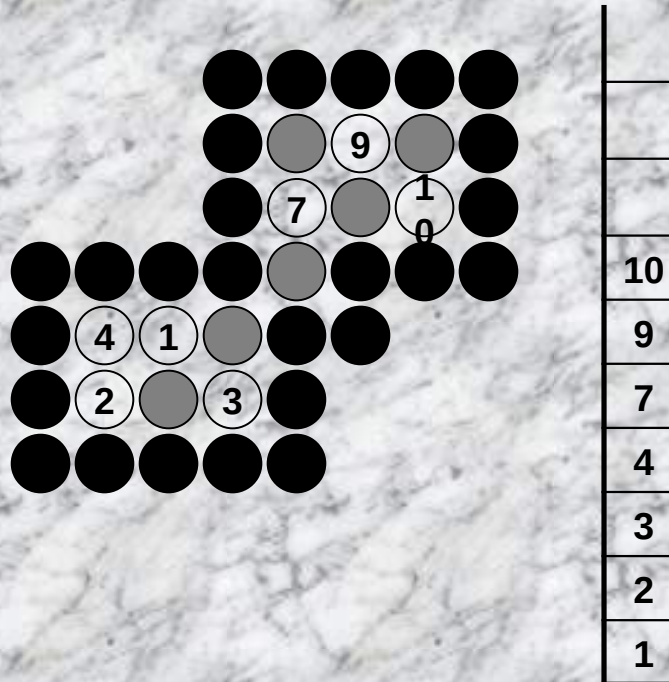
Boundary Fill Algorithm

8-connected (Example)



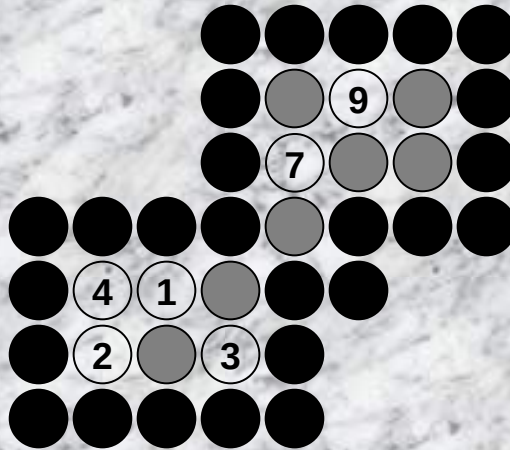
Boundary Fill Algorithm

8-connected (Example)



Boundary Fill Algorithm

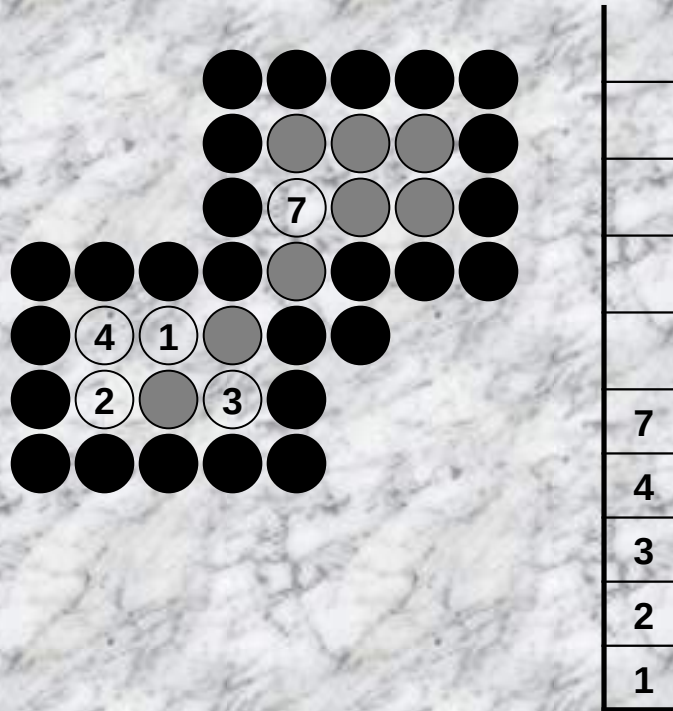
8-connected (Example)



9
7
4
3
2
1

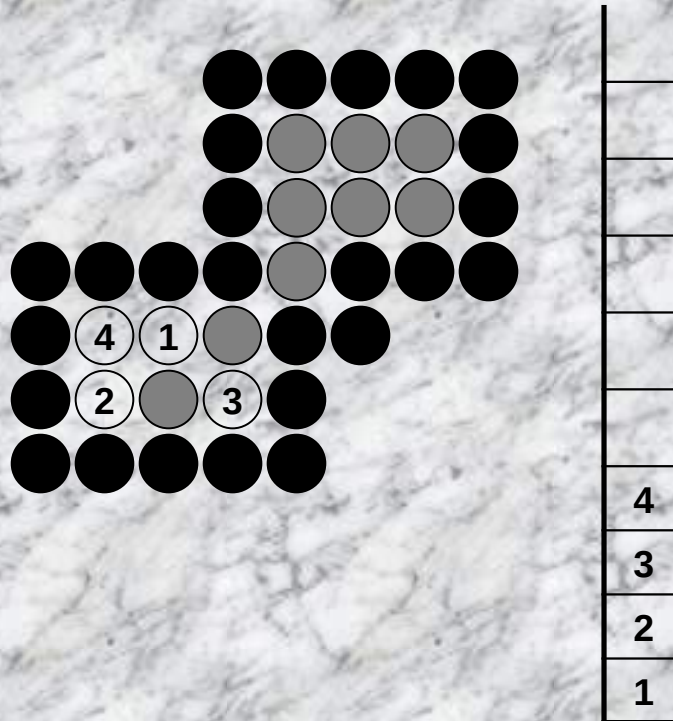
Boundary Fill Algorithm

8-connected (Example)



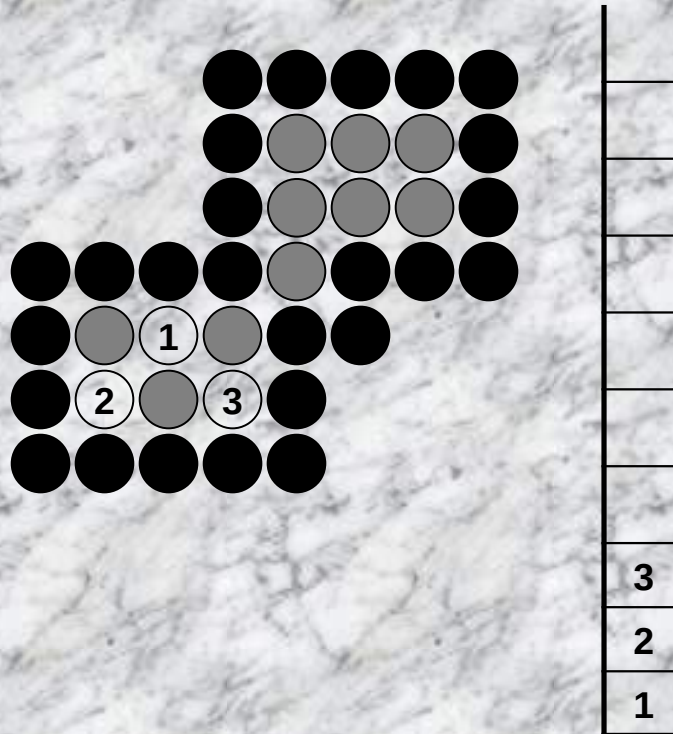
Boundary Fill Algorithm

8-connected (Example)



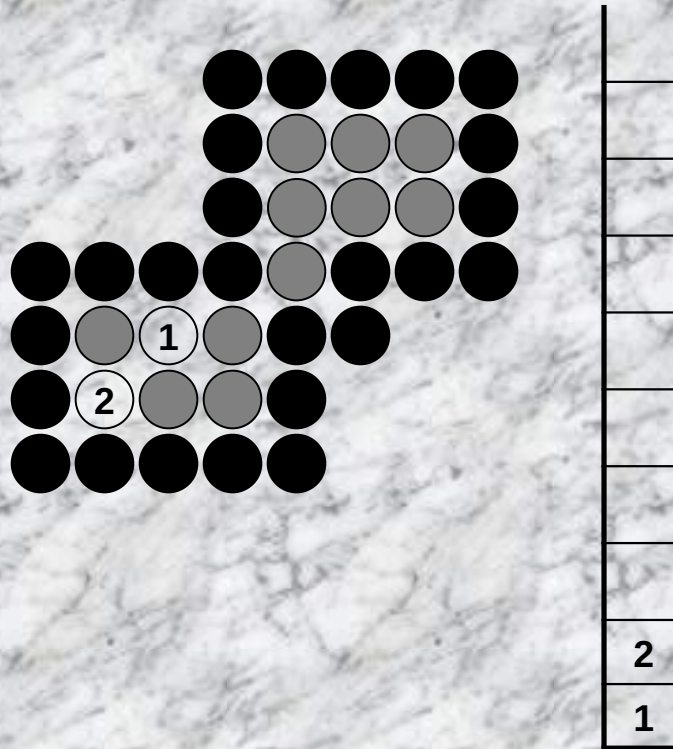
Boundary Fill Algorithm

8-connected (Example)



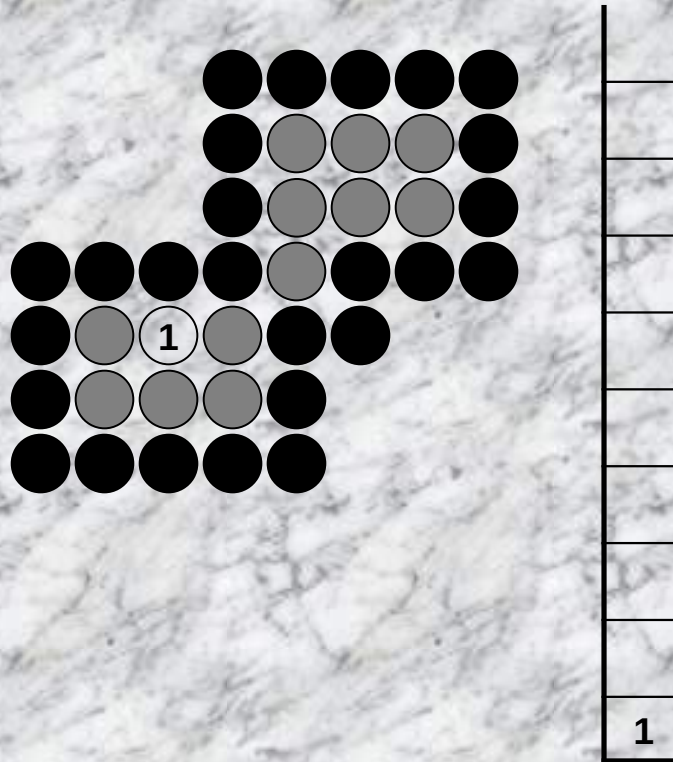
Boundary Fill Algorithm

8-connected (Example)



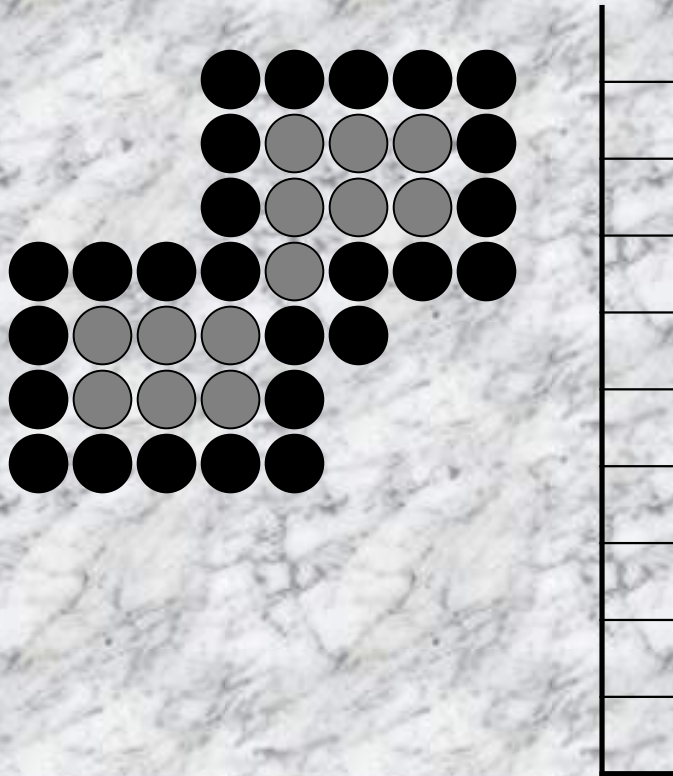
Boundary Fill Algorithm

8-connected (Example)



Boundary Fill Algorithm

8-connected (Example)



Boundary Fill Algorithm

Since the previous procedure requires considerable stacking of neighboring pixels, more **efficient methods** are generally employed.

These methods (**Span Flood-Fill**) **fill horizontal pixel spans across scan lines**, instead of proceeding to 4-connected or 8-connected neighboring pixels.

Then we need only stack a beginning position for each horizontal pixel spans, instead of stacking all unprocessed neighboring positions around the current position.

Polygon Filling Algorithm

```
void boundary_fill (x, y, fc, bc){  
  
    if(getpixel(x,y) != bc && getpixel(x,y) !=fc){  
        putpixel(x,y,fc);  
        boundary_fill(x+1, y, fc,bc);  
        boundary_fill(x, y+1, fc,bc);  
        boundary_fill(x-1, y, fc,bc);  
        boundary_fill(x, y-1, fc,bc);  
    }  
}
```

(x,y): current pixel

fc: new color

bc: boundary color

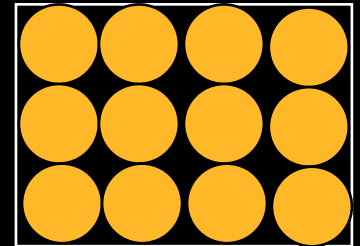
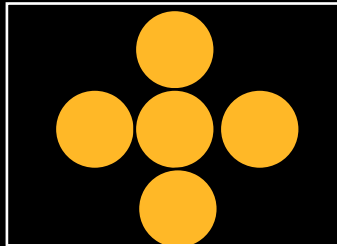
getpixel(x,y): returns color of (x,y)

putpixel(x,y,c): paint (x,y) with color c

Polygon Filling

- ◆ **Seed Fill:**

- **Flood Filling:** starts from seed, old color replace by new color. Here old color is black and new color is yellow border color is white



Flood fill Function

```
void flood_fill(int x,int y,int newc,int oldc){  
    if(getpixel(x,y)==oldc){  
  
        putpixel(x,y,newc);  
        flood_fill(x+1,y,newc,oldc);  
        flood_fill(x,y+1,newc,oldc);  
        flood_fill(x-1,y, newc,oldc);  
        flood_fill(x,y-1, newc,oldc);  
  
    }
```

The End